

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2115 - Programación como herramienta para la ingeniería

Modelos predictivos de aprendizaje supervisado

Profesor: Hans Löbel

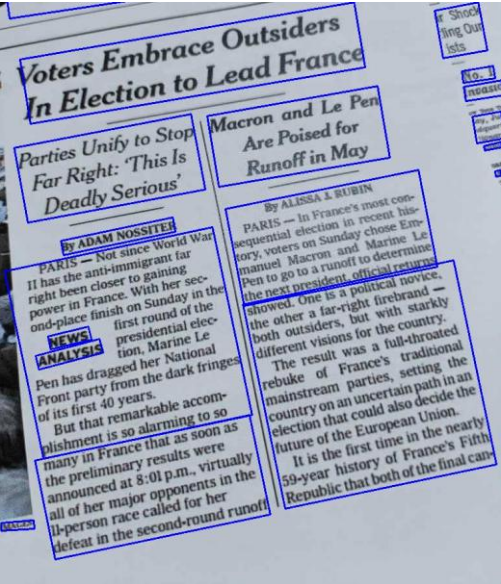
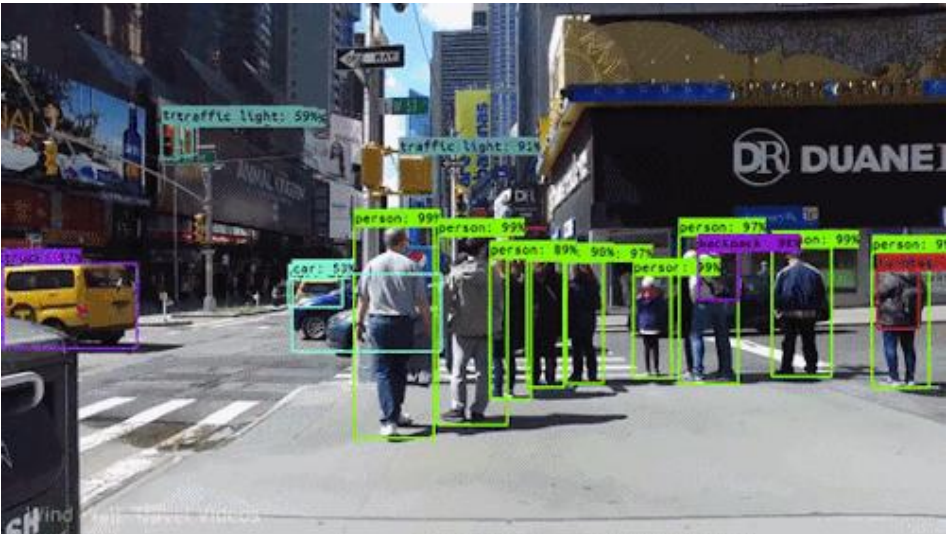
¿Cómo puedo saber si en una foto hay un perro o un gato?
(usando un computador y sin preguntarle a un asistente, obviamente)

¿



?





Generated Question	Answer
How many bears are laying on the ice?	"two"
What are the two animals laying on the ice?	"bears"
What are the bears doing?	"laying down"
Two bears are laying down on what?	"ice"
Where are the bears laying?	"on the ice"
Are the bears on the ice?	"yes"



A dog wearing a Superhero outfit with red cape flying through the sky

En simple, **Machine Learning** será para nosotros el estudio sobre **cómo hacer predicciones a partir de datos**

- Adecuado para problemas donde la programación “tradicional” no es posible (reglas, algoritmos o procedimientos explícitos).
- Éxito o fracaso depende del rendimiento en ejemplos no “vistos” previamente.
- Metodologías y protocolos para “entrenar” buenos modelos son el foco de este capítulo.



Con respecto a la herramienta específica para ML, en este capítulo nos centraremos en **scikit-learn**

- Implementa gran cantidad de algoritmos y modelos predictivos, basados en **Machine Learning**.
- Clasificación, regresión, *clustering*, reducción de dimensionalidad, selección de modelos y más.
- Permite una fácil integración con Pandas y librerías de visualización.

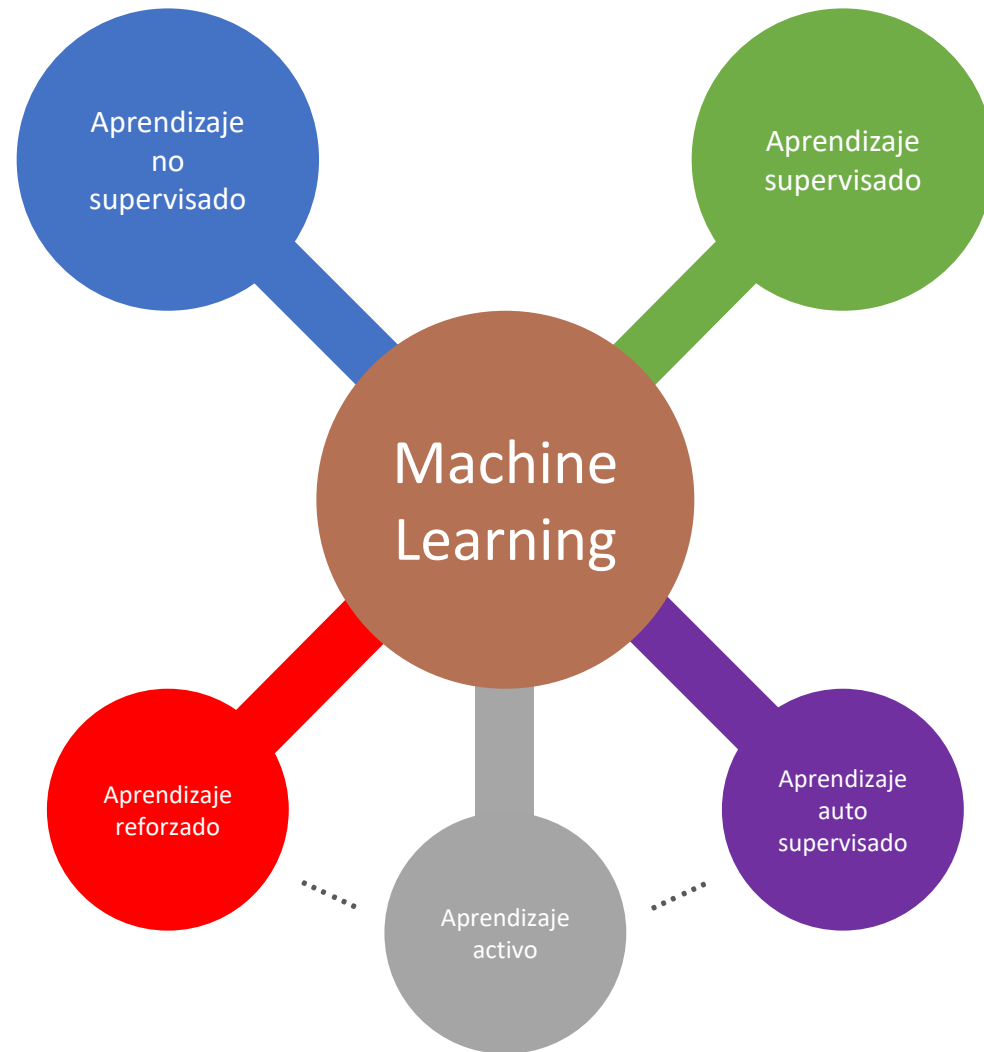


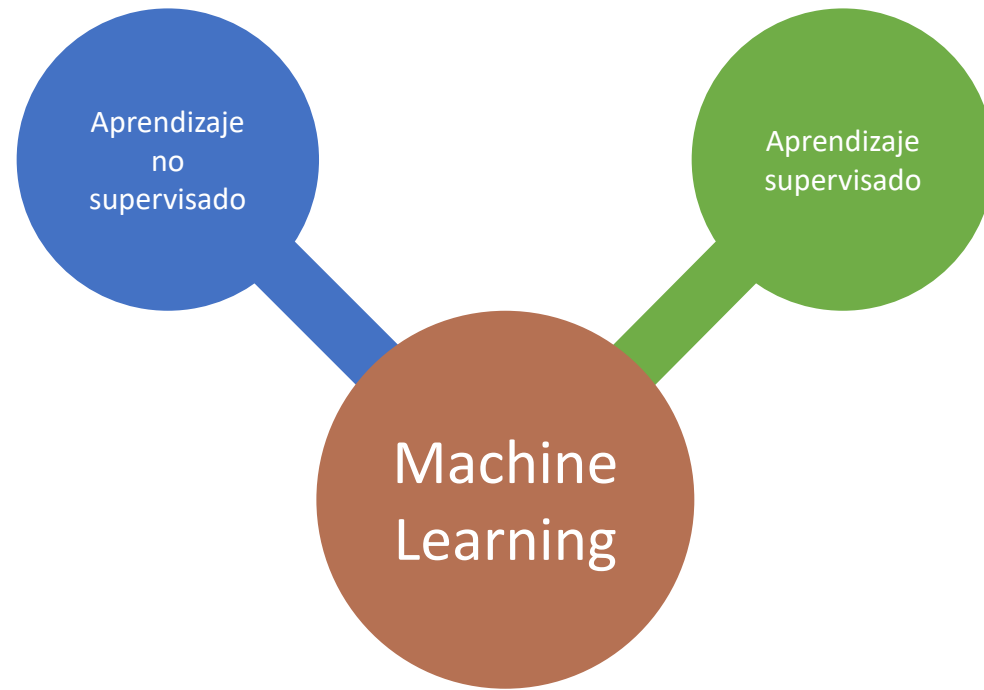
Antes de revisar *scikit-learn* y el tipo de problemas en este capítulo, necesitamos una breve introducción a los elementos centrales de **Machine Learning**

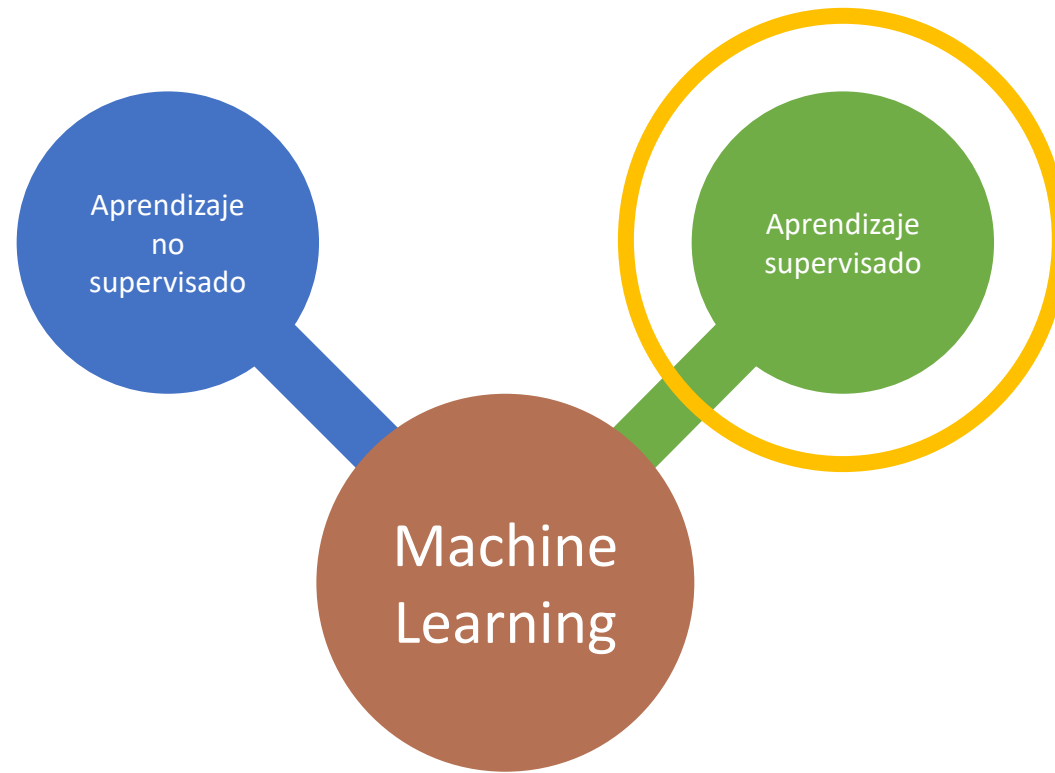
Ahora con algo más de detalle, ¿Qué es Machine Learning?

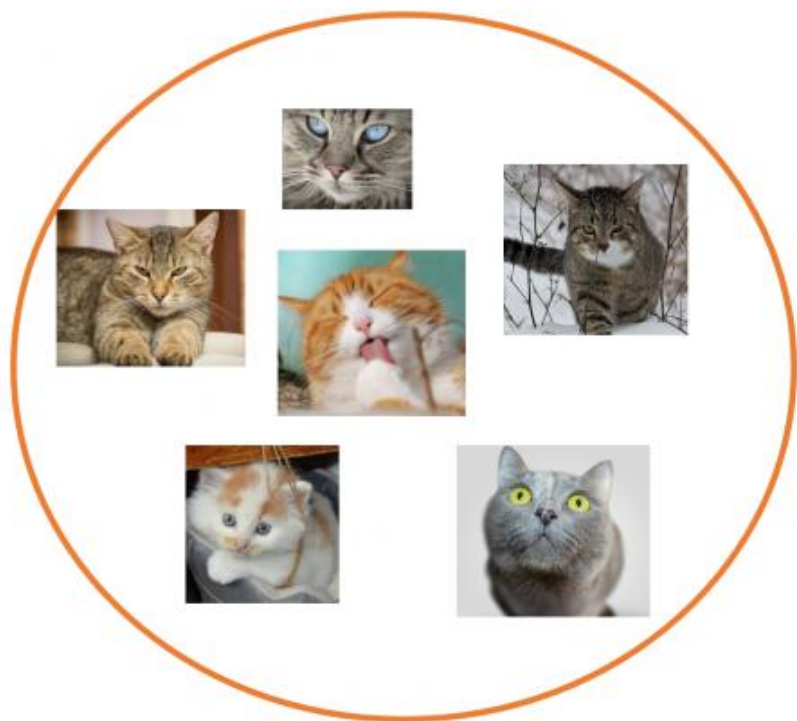
- Técnicas computacionales que usan **datos** para realizar **tareas predictivas** (clasificación, regresión, reducción de dimensionalidad, clustering, etc.)
- Típicamente, estas técnicas (modelos, algoritmos, etc.) mejoran su **rendimiento** predictivo a través de la experiencia → **aprendizaje** desde los **datos**.
- Foco es distinto a la estadística: técnicas de ML buscan realizar **tareas/predicciones** con la mayor precisión posible, más que entender el fenómeno subyacente.









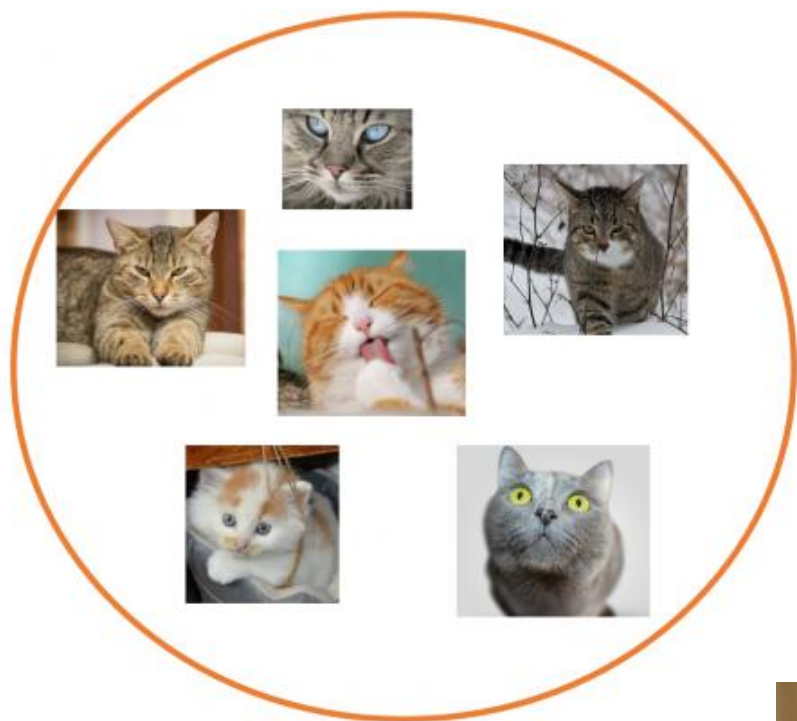


Cat



Dog





Cat



Dog

¿



?

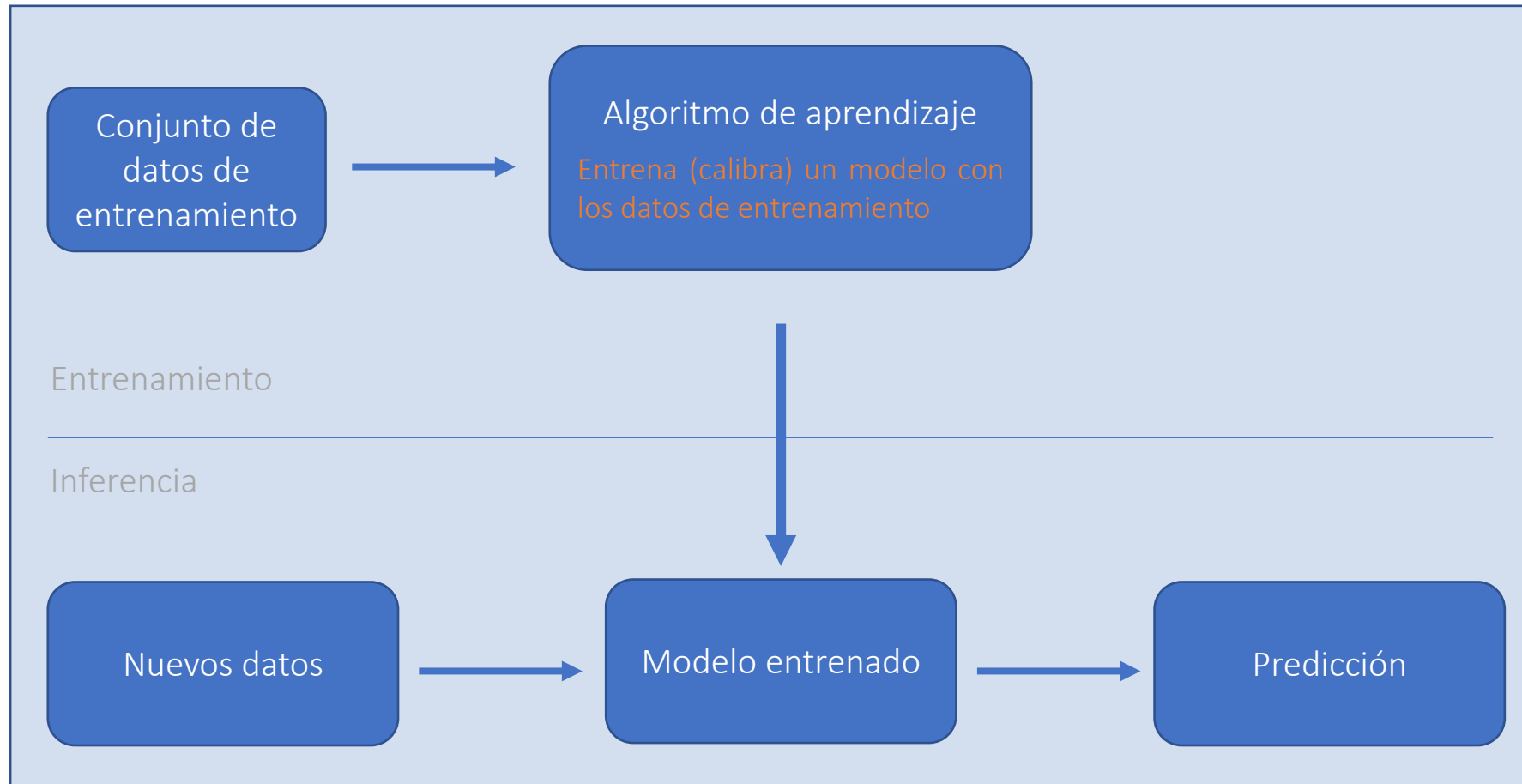
Como problema guía para lo que viene, estudiaremos la siguiente pregunta:

¿Cómo puedo estimar el ingreso anual promedio per cápita para los habitantes de una ciudad?

Como insumo para el estudio, tenemos un archivo `.csv` con datos diversos para múltiples ciudades

Ciudad	Población	Región geográfica	Superficie (km²)	Densidad poblacional por km²	Actividad económica principal	Ingreso anual promedio per cápita en dólares
Metropolis	3.500.000	Centro	600	5.833	Servicios	\$45.000
Rivertown	1.200.000	Centro	150	8.000	Industria	\$35.000
Coastline	2.800.000	Costa	300	9.333	Turismo	\$40.000
Highland	900.000	Norte	350	2.571	Minería	\$30.000
Greenfield	600.000	Sur	400	1.500	Agricultura	\$28.000
Sunnyside	750.000	Costa	500	1.500	Turismo	\$32.000
Frostville	300.000	Sur	200	1.500	Industria	\$27.000
Eastbank	1.100.000	Centro	250	4.400	Comercio	\$36.000
Westwood	2.300.000	Costa	450	5.111	Servicios	\$42.000
Clearwater	500.000	Sur	180	2.777	Agricultura	\$29.000
Ironforge	800.000	Norte	190	4.211	Industria	\$40.000
Lakeview	950.000	Centro	220	4.318	Servicios	\$47.000
Cedarwood	410.000	Sur	300	1.367	Agricultura	\$25.000
Newhaven	1.500.000	Costa	280	5.357	Servicios	\$70.000
Pinecrest	670.000	Norte	330	2.030	Minería	\$33.000
Stonebridge	390.000	Centro	210	1.857	Industria	\$11.000

(casi) Todas las técnicas de ML usan el mismo esquema de procesamiento



Datos de entrenamiento son una muestra finita del universo de posibles datos

- Deben ser lo más representativos posibles del problema a resolver
- Cantidad y variabilidad son los aspectos más importantes
- Identificadores no deben formar parte de este conjunto

Datos de
Entrenamiento

Población	Región geográfica	Superficie (km ²)	Densidad poblacional por km ²	Actividad económica principal	Ingreso anual promedio per cápita en dólares
3.500.000	Centro	600	5.833	Servicios	\$45.000
1.200.000	Centro	150	8.000	Industria	\$35.000
2.800.000	Costa	300	9.333	Turismo	\$40.000
900.000	Norte	350	2.571	Minería	\$30.000
600.000	Sur	400	1.500	Agricultura	\$28.000
750.000	Costa	500	1.500	Turismo	\$32.000
300.000	Sur	200	1.500	Industria	\$27.000
1.100.000	Centro	250	4.400	Comercio	\$36.000

Técnicas de ML trabajan sobre datos **multidimensionales**

- Cada dato es representado por una serie de **características** = *features* = atributos = variables.
- Siempre buscaremos **predecir algo** a partir de estas características.
- Muchas veces, los datos poseen además una **etiqueta** = *label* = rótulo, que típicamente buscaremos predecir.

Datos de
Entrenamiento

Características					Etiqueta
Población	Región geográfica	Superficie (km²)	Densidad poblacional por km²	Actividad económica principal	Ingreso anual promedio per cápita en dólares
3.500.000	Centro	600	5.833	Servicios	\$45.000
1.200.000	Centro	150	8.000	Industria	\$35.000
2.800.000	Costa	300	9.333	Turismo	\$40.000
900.000	Norte	350	2.571	Minería	\$30.000
600.000	Sur	400	1.500	Agricultura	\$28.000
750.000	Costa	500	1.500	Turismo	\$32.000
300.000	Sur	200	1.500	Industria	\$27.000
1.100.000	Centro	250	4.400	Comercio	\$36.000



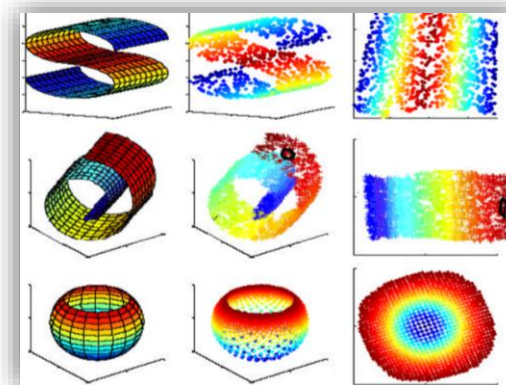
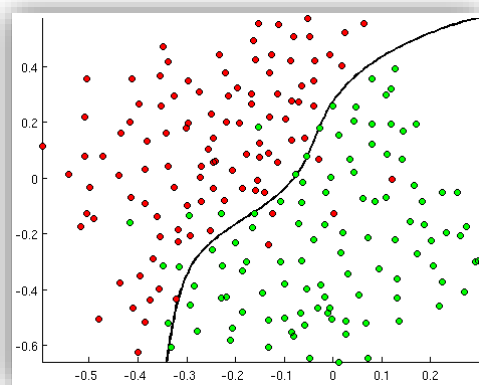
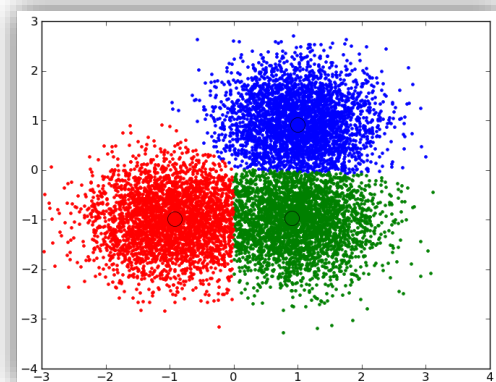
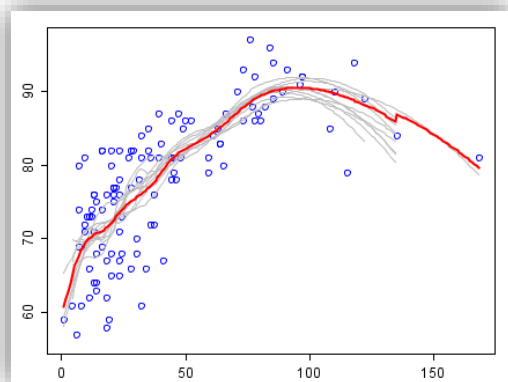
Próxima sesión veremos casos donde no hay etiqueta

Técnicas de ML trabajan sobre datos multidimensionales

- Cada dato del conjunto de entrenamiento puede considerarse como un vector.
- La cantidad de características define la dimensionalidad del vector.
- El espacio donde viven los vectores se conoce como espacio de características.

Características					Etiqueta
Población	Región geográfica	Superficie (km²)	Densidad poblacional por km²	Actividad económica principal	Ingreso anual promedio per cápita en dólares
3.500.000	Centro	600	5.833	Servicios	\$45.000
1.200.000	Centro	150	8.000	Industria	\$35.000
2.800.000	Costa	300	9.333	Turismo	\$40.000
900.000	Norte	350	2.571	Minería	\$30.000
600.000	Sur	400	1.500	Agricultura	\$28.000
750.000	Costa	500	1.500	Turismo	\$32.000
300.000	Sur	200	1.500	Industria	\$27.000
1.100.000	Centro	250	4.400	Comercio	\$36.000

Cada dato puede verse como un vector/punto en el espacio de características



Esto hace que datos y tareas puedan interpretarse de manera más intuitiva desde un punto de vista geométrico

Teniendo lo básico cubierto, podemos revisar como
usar *scikit-learn* para resolver problemas predictivos

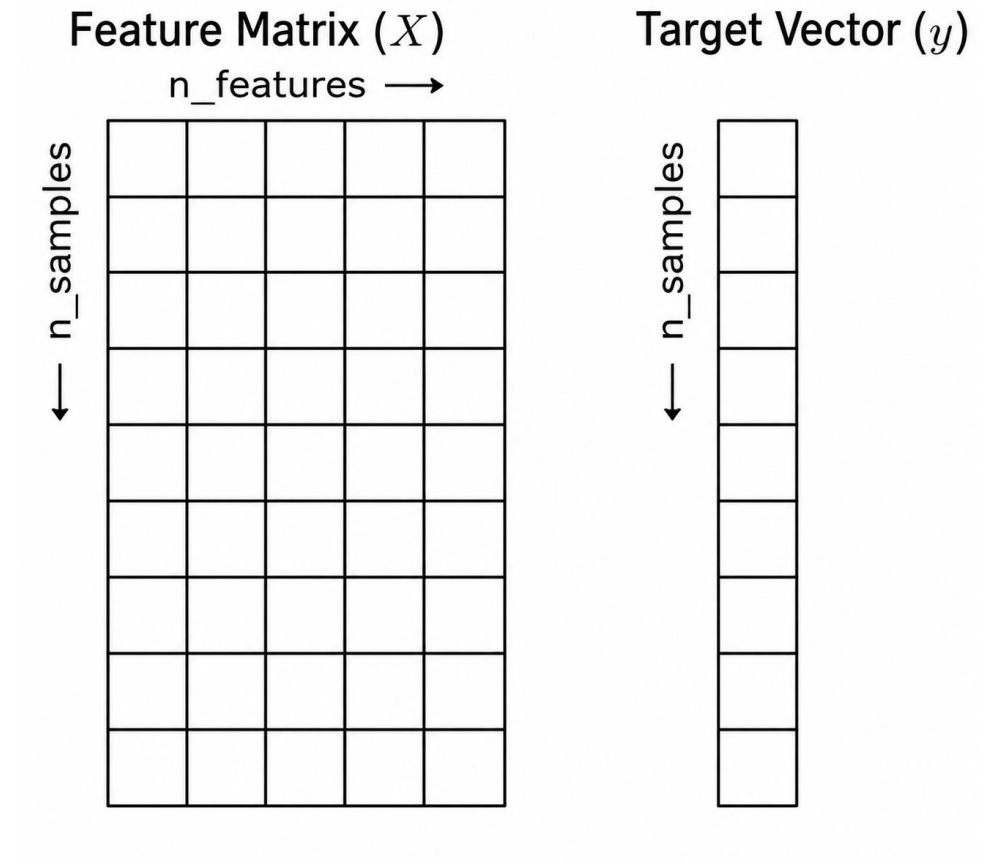
Scikit-learn será nuestra herramienta principal para este capítulo

- scikit-learn es el módulo para ML más conocido y utilizado en Python.
- Su principal atractivo es una interfaz limpia, uniforme y simple, que facilita la exploración y permite la integración con otros paquetes, como Pandas.
- Posee además de una completa documentación en línea (<https://scikit-learn.org/>).



Esquema de datos de **scikit-learn** es similar a Pandas

- Datos son representados por **matriz de características** y, si la tarea es supervisada, un **vector objetivo** de etiquetas.
- Características se almacenan en matriz **X**, de tamaño $[n_samples, n_features]$ (típicamente será un **DataFrame**).
- El vector objetivo **y** contiene la etiqueta para cada ejemplo. Tiene tamaño $[n_samples, 1]$ (típicamente será un **Series**).



En general, el flujo de uso de scikit-learn es siempre el mismo

1. Obtener o generar matriz **X** y vector **y** → Preparar los datos (ver la materia antes de tirarse directo a los ejercicios)

Población	Región geográfica	Superficie (km ²)	Densidad poblacional por km ²	Actividad económica principal	Ingreso anual promedio per cápita en dólares
3.500.000	Centro	600	5.833	Servicios	\$45.000
1.200.000	Centro	150	8.000	Industria	\$35.000
2.800.000	Costa	300	9.333	Turismo	\$40.000
900.000	Norte	350	2.571	Minería	\$30.000
600.000	Sur	400	1.500	Agricultura	\$28.000
750.000	Costa	500	1.500	Turismo	\$32.000
300.000	Sur	200	1.500	Industria	\$27.000
1.100.000	Centro	250	4.400	Comercio	\$36.000

Preparación de los datos tiene un foco distinto al capítulo pasado

- Los datos de entrenamiento deben representar bien el problema.
- Conviene incorporar la menor cantidad posible de supuestos.
- Es más importante preservar la información disponible que descartar datos imperfectos.
- Faltantes y outliers pueden tratarse posteriormente usando los datos válidos.
- Para evitar errores, sigan un protocolo claro de codificación, división y transformación.



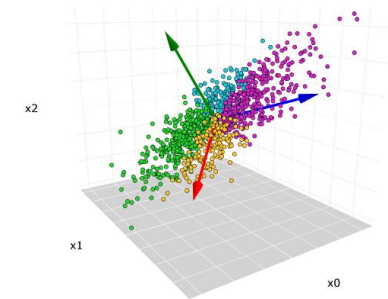
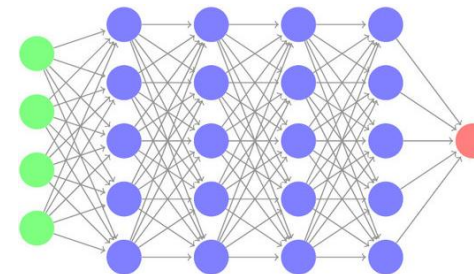
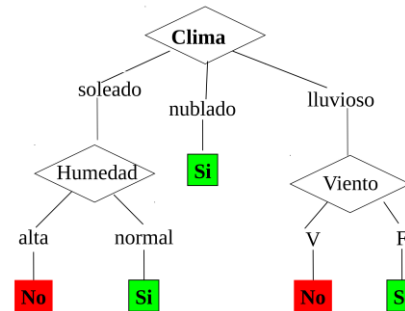
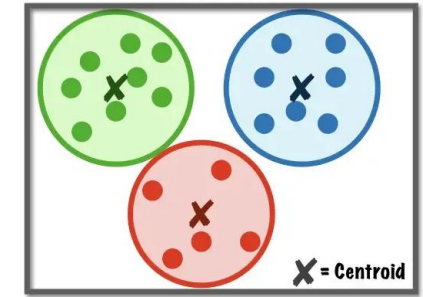
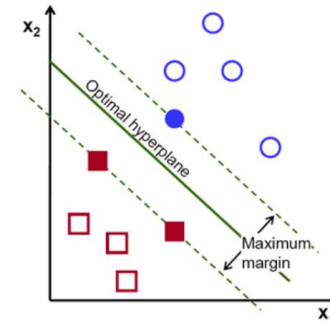
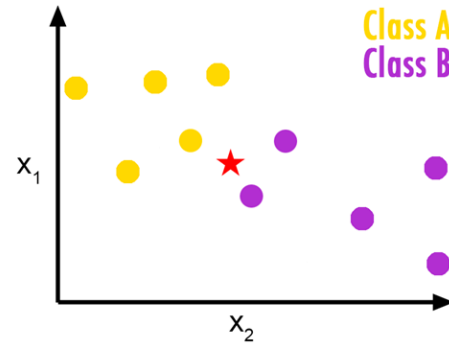
En general, el flujo de uso de scikit-learn es siempre el mismo

1. Obtener o generar matriz X y vector $y \rightarrow$ Preparar los datos (ver la materia antes de tirarse directo a los ejercicios)
2. Elegir modelos **adecuados** dependiendo si se hace **clasificación o regresión**, importando la clase correspondiente desde *sklearn*.
3. Entrenar los modelos llamando al método **fit(X, y)**.



Una vez listos los datos, podrán utilizar múltiples modelos predictivos, dependiendo de la tarea a resolver

- k-NN
- Regresiones SVM
- Árboles de decisión
- Ensamblés
- Redes neuronales
- y más...

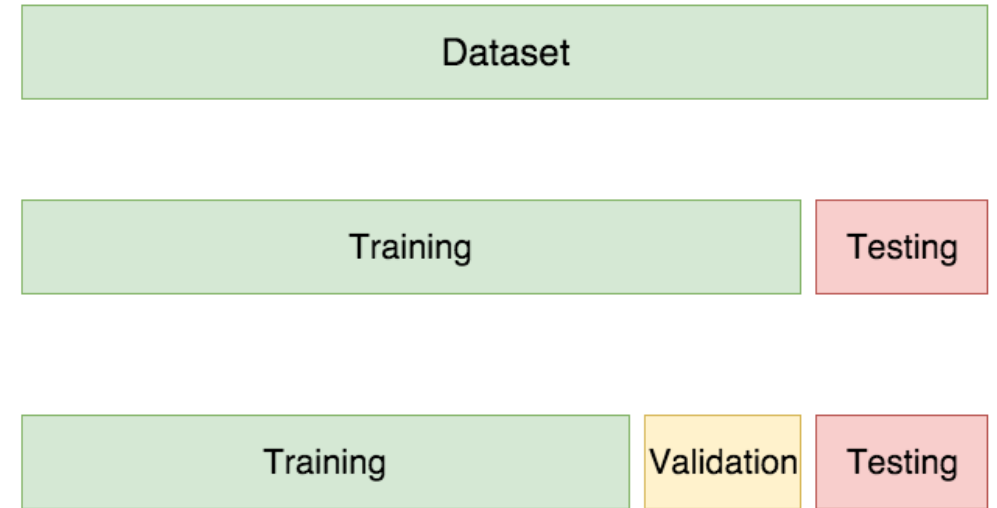


En general, el flujo de uso de scikit-learn es siempre el mismo

1. Obtener o generar matriz X y vector $y \rightarrow$ Preparar los datos (ver la materia antes de tirarse directo a los ejercicios)
2. Elegir modelos adecuados dependiendo si se hace clasificación o regresión, importando la clase correspondiente desde *sklearn*.
3. Entrenar los modelos llamando al método `fit(X, y)`.
4. Seleccionar el mejor modelo (ver la materia antes de tirarse directo a los ejercicios).

La elección del mejor modelo requiere rigurosidad en el procedimiento

- Lo primero que necesitamos es una manera de medir el rendimiento de un modelo, para lo que debemos definir dos cosas: una métrica de rendimiento y conjuntos de validación y prueba.
- La métrica de rendimiento estará dada generalmente por la tarea a resolver (scikit-learn ofrece muchas en el módulo `sklearn.metrics`).
- Conjuntos de validación y prueba deben venir de la misma distribución de datos que el entrenamiento (nuevamente scikit-learn nos facilita el trabajo con el método `train_test_split`).
- Acá es también es fundamental revisar la material para hacer esto correctamente.



En general, el flujo de uso de scikit-learn es siempre el mismo

1. Obtener o generar matriz X y vector $y \rightarrow$ Preparar los datos (ver la materia antes de tirarse directo a los ejercicios)
2. Elegir modelos adecuados dependiendo si se hace clasificación o regresión, importando la clase correspondiente desde sklearn.
3. Entrenar los modelos llamando al método `fit(X, y)`.
4. Seleccionar el mejor (ver la materia antes de tirarse directo a los ejercicios).
5. Aplicar el modelo entrenado en nuevos datos, usando el método `predict()` y medir el rendimiento final.

En resumen...

- ML se centra en algoritmos/modelos que aprenden de los datos para resolver una tarea
- scikit-learn permite hacer ML en Python de manera práctica y rápida
- (casi) Todas las técnicas de ML funcionan de la misma manera
- Qué técnica usar dependerá de la tarea y los datos disponibles, pero siempre se debe **decidir en base a métricas de rendimiento**
- Todo esto y mucho más en los cursos IIC2613 – Inteligencia Artificial, **ICT3115 – Sistemas Urbanos Inteligentes** e IIC3697 – Aprendizaje Profundo.

Pontificia Universidad Católica de Chile
Escuela de Ingeniería
Departamento de Ciencia de la Computación



IIC2115 - Programación como herramienta para la ingeniería

Modelos Predictivos basados en Machine Learning

Profesor: Hans Löbel